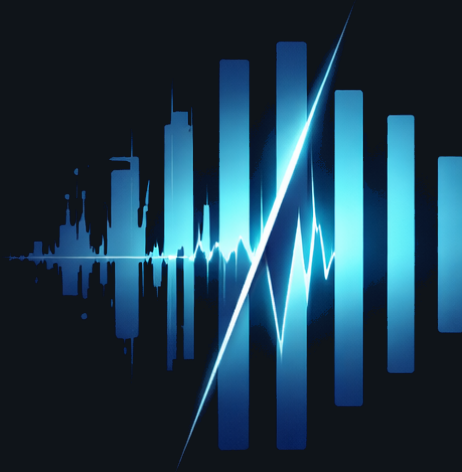




NON SEQUITUR PUBLISHING · WHITE PAPER

RF Skip Jack

Mission-State Custody Across Intermittent Links

Race conditions, deterministic personas, and the orchestration layer above Layer 7

AI on the Degraded Edge — Volume II

PREPARED BY: Non Sequitur Publishing — Architecture & Engineering
DOCUMENT VERSION: v0.2 (Pre-release, draft for review)
DATE: May 2026
CLASSIFICATION: Public — Defensive Disclosure
CANONICAL URL: nsq.tech/pubs/rf-skipjack
SERIES: AI on the Degraded Edge

© 2026 Non Sequitur Publishing.

All operational scenarios in this paper are fictitious; see the *Fictitious Use Case* notice on the next page.

Contents

Mission-State Custody Across Intermittent Links	1
§1 Introduction	2
§2 Operational Context	5
§3 The Race-Condition Problem Set	7
§4 Capabilities Required	11
§5 Prior Art and Where It Falls Short	16
§6 The Orchestration Pattern	19

““markdown # RF Skip Jack

Mission-State Custody Across Intermittent Links

Race conditions, deterministic personas, and the orchestration layer above Layer 7

AI on the Degraded Edge — Vol. II

Author: Non Sequitur Publishing — Architecture & Engineering **Version:** v0.2 (Pre-release, draft for review) **Date:** May 2026 **Cover banner:** Public — Defensive Disclosure **Series predecessor:** *AI on the Degraded Edge — Vol. I* [1]

Provenance & Synthesis

The architectural patterns, race-condition formalizations, and capability requirements in this paper are synthesized from publicly observable distributed-systems literature, mission-management standards, and the well-known operational properties of degraded-link environments. They are not derived from any single customer deployment, classified program, protected implementation, or specific procurement context. The orchestration substrate this paper describes is a doctrinal framework; specific implementations are matters for engineering specification outside this paper’s scope.

Strategic positioning

This paper occupies a doctrinal-framework position. The capabilities and semantics defined here apply to any deterministic AI orchestration substrate operating under degraded mission-state conditions, whether the operational class is maritime, space, expeditionary edge, autonomous mission systems, disconnected AI operations, or industrial edge environments. The architectural doctrine is operationally neutral with respect to physical regime; the binding structure is the operational property of the link and the operator-bounded-autonomy requirement, not the specific deployment context.

Abstract

Operational environments characterized by intermittent, low-bandwidth, high-latency, and contended links between edge-collection / edge-execution nodes and a converging mission-state authority create a class of race conditions that existing transport-layer products, file-transfer protocols, and battle-management systems do not adequately address. This paper formalizes that race-condition class across seven distinct race types, specifies a complementary ten-capability envelope that any solution must realize, and presents an above-Layer-7 orchestration pattern using deterministic persona-decomposed agents under a human governance and curation framework (HGC³AE² — Human Governance and Curation of Adaptive Autonomous Agentic Entities) as one realization. The architecture is positioned as applicable across multiple operational classes including allied maritime forces in degraded-SATCOM regions, military space command and control, civil deep-space mission operations, expeditionary terrestrial deployments, autonomous mission systems, disconnected AI operations, and industrial edge environments. Prior art across delay-tolerant networking (Bundle Protocol v7, RFC 9171), WAN acceleration appliances, UDP file-transfer protocols, conflict-free replicated data types, and battle-management software is surveyed; each is shown to address part of the capability envelope, none to address the full set. The contribution is the integration of these capabilities into a complementary envelope above Layer 7 specifically for mission-state custody across intermittent links, governed by operator-bounded autonomy, and extended in this volume to a proto-specification of deterministic persona contracts, custody object model, governance-state arbitration, and freshness-aware mission semantics. A validation framework based on link-condition simulation and operator-in-the-loop measurement is sketched. This paper is the second in the *AI on the Degraded Edge* series and serves as the defensive-disclosure vehicle for the conceptual frame and its doctrinal extension.

Keywords: degraded-link operations, intermittent connectivity, mission-state custody, race condition, deterministic AI orchestration, above-L7 transport, governed autonomy, delay-tolerant networking, persona-decomposed agents, deterministic persona contracts, custody object lifecycle, governance-state arbitration, freshness-aware mission semantics.

§1 Introduction

1.1 Motivation across operational classes

Consider three operational scenarios that share a common structural problem despite operating in very different physical regimes:

An allied maritime force is operating across a degraded-SATCOM region. Vessels collect intelligence, surveillance, and reconnaissance data at the edge; a shore-side mission-management center directs tasking and ingests collected products. The link between vessel and shore is a shared time-division-multiplexed satellite channel with multi-second latency, intermittent availability under weather and contention, and bandwidth budgets measured in single-digit megabits per second per user.

A military space command is managing command-and-control traffic across an orbital relay constellation. Tasking decisions made at a ground station propagate through one or more relay hops to assets on orbit;

sensor products and asset state propagate back. Link windows are scheduled and finite; contention with other ground stations is endemic; the operator-decision loop on the ground depends on a converged view of asset state that is, by physical necessity, always slightly out of date.

A civil space mission is running a custody-aware file transfer over a deep-space link spanning light-minutes of one-way latency. The link is intermittent by orbital geometry. The ground-based mission operations center must reason about the in-flight state of artifacts in a domain where retransmit cycles are measured in hours and where naïve transport assumptions about acknowledgement timing fail entirely.

Beyond these three illustrative scenarios, the same structural problem arises in autonomous mission systems where on-platform decision-making must compose with operator authority over an intermittent link, in disconnected AI inference operations where edge inference outputs must be reconciled with shore-side mission models, and in industrial edge environments where remote process-control systems operate across constrained backhaul. These cases — and the broader class enumerated in §2 — share the same structural property: an edge-collection / edge-execution environment is in continuous, partial, latency-bound dialogue with a converging mission-state authority, and the dialogue is itself subject to race conditions that arise from the link’s intermittency, the multi-second clock drift between endpoints, and the operator’s need for a fresh, consistent view at decision time.

1.2 The mission-state-custody problem

We define *mission-state custody* as the property that, at any given moment, both ends of an intermittent link possess a mutually consistent and reliably-attributable view of the artifacts and decisions that constitute the mission. Custody requires:

- A globally consistent identity for every artifact (who produced it, when, what its hash is).
- A globally consistent state machine for every artifact’s lifecycle (created, enqueued, in-flight, received, acknowledged, durable).
- A globally consistent ordering for events that mutate mission state, even when the wall-clocks at the two endpoints disagree by seconds.
- An operator-visible freshness annotation on every cached object, so decisions are never made on stale state without the operator knowing it is stale.
- A reconciliation discipline for the case where both ends have written to the same logical state during link partition.

When custody fails, operators make decisions on stale or incorrect state; tasking is issued against assumptions that no longer hold; collected products are lost or duplicated; and the mission-management surface becomes untrustworthy at exactly the moments when trust is most needed.

1.3 Why this matters now

Three converging factors make this problem class urgent:

First, low-earth-orbit constellation proliferation [UNVERIFIED — citation likely: LEO measurement papers 2023-2025] has expanded the deployment of intermittent-link mission environments well beyond traditional

military maritime use. Disaster response, scientific expedition, commercial constellation management, and lunar surface operations are all now operating on links with the same fundamental properties.

Second, edge-AI inference workloads at the maritime, airborne, and orbital edge generate artifacts whose disposition the mission-management surface needs to reason about — inference results, model updates, sensor-fusion outputs — and these artifacts must move across the intermittent link with custody intact.

Third, multi-domain operations require bidirectional state convergence across domains (e.g., maritime collection feeding space-based cueing feeding terrestrial decision-making) where each domain’s link characteristics differ but all share the intermittent-link structural problem.

1.4 Contribution claims

This paper makes four contribution claims, each of which serves as a defensive-disclosure anchor:

Claim 1. A formal definition of a class of race conditions that arise in mission-state custody over intermittent links. We enumerate seven distinct race conditions (§3) with formal participants, racing operations, failure modes, and invariants violated. The race-condition class is invariant across operational domains (maritime, space, expeditionary, autonomous mission systems, disconnected AI, industrial edge).

Claim 2. A demonstration that existing transport-layer solutions, file-transfer protocols, eventual-consistency databases, and battle-management systems do not address this class in full (§5). Each addresses part; none addresses the whole.

Claim 3. Specification of an above-Layer-7 orchestration pattern using deterministic persona-decomposed agents (§6) — Forwarder, Custody Manager, Fragmentation Manager — as one realization of the capability envelope required by Claim 1. The pattern is extended in this volume to a proto-specification of (a) deterministic persona contracts (§6.10), (b) the custody object lifecycle and conceptual structure (§6.11), (c) governance-state arbitration under uncertainty (§6.12), and (d) freshness-aware mission semantics (§6.13).

Claim 4. A governance framework — HGC³AE² (Human Governance and Curation of Adaptive Autonomous Agentic Entities) — for operator-bounded autonomy of the autonomous components, enabling defensive replay-ability and operator-revocable execution tokens such that the autonomous layer remains the operator’s instrument rather than its substitute.

1.5 Roadmap

§2 characterizes the operational context across multiple customer classes and identifies the unifying structural properties. §3 — the load-bearing problem-set section — formalizes the seven race conditions. §4 — the load-bearing requirements section — specifies the ten-capability complementary envelope that any solution must realize. §5 surveys prior art and identifies the structural gap that lives above Layer 7. §6 presents the orchestration pattern as one realization of the envelope, with explicit traceability from each design element back to the capabilities of §4, and includes the proto-specification of deterministic persona contracts, custody object model, governance arbitration, and freshness semantics. §7 sketches the validation framework. §8 discusses applicability boundaries and limitations. §9 concludes and names the forward series direction.

§2 Operational Context

2.1 Characterizing degraded-link operations

The operational contexts that fall within scope of this paper share four link-level characteristics:

- **Low bandwidth.** Effective throughput between edge and converging mission-state authority is in the single-digit megabits per second per user, often less. Bandwidth is contended with other users on a shared transport.
- **Multi-second latency.** One-way latency between endpoints is measured in seconds, not milliseconds. Round-trip latency for typical TCP-based protocols breaks the assumptions that L4 timing depends on.
- **Intermittent availability.** Link windows are bounded by physical phenomena — orbital geometry, weather attenuation, line-of-sight obstruction, scheduled time-division slots — and are not under the control of either endpoint.
- **Contention.** The link is shared across multiple concurrent users, each with its own priority class, and the share each user receives at any instant is non-stationary.

When all four properties obtain simultaneously, the operational mode is what we term *degraded-link operations*.

2.2 Mission classes dependent on bidirectional sync

Within degraded-link operations, the following mission classes specifically depend on bidirectional state synchronization rather than unidirectional bulk transfer:

- **ISR collection-and-tasking loops** — sensor products move from edge to shore; tasking and re-tasking decisions move from shore back to edge; both directions must converge on a consistent view of what has been collected and what has been requested.
- **Command-and-control / mission-management portals** — decisions made by operators at the converging authority node must propagate to edge assets; asset state must propagate back; the portal is the surface where both views converge.
- **Distributed sensor fusion** — observations from multiple edge nodes feed a fusion process; fusion outputs feed tasking decisions; tasking propagates back to edge sensors. The fusion process requires consistent input ordering.

2.3 The current state of the art

Today, the operational practice across these mission classes is some combination of:

- **Ship-dump-on-return.** Edge nodes accumulate artifacts locally and transfer them in bulk when a sufficiently wide link window is available, accepting that the mission-management surface lags edge reality by hours or days.

- **Partial sync with manual reconciliation.** Operators manually compare edge and shore views and resolve discrepancies through voice or chat coordination, accepting the cognitive burden as the cost of operating in this regime.
- **Out-of-band coordination channels.** Critical decisions are communicated through low-bandwidth, high-reliability side channels (voice, short-message text) while the bulk data path runs separately and asynchronously.

Each of these is a workaround for the absence of a custody-aware orchestration layer. Each preserves the mission at the cost of operator cognitive load, decision latency, and risk of acting on stale state.

2.4 Why the operator-decision loop is the binding constraint

The mission-state-custody problem is, ultimately, a problem about the operator's decision loop. Operators on the converging mission-state authority node act on a view of mission state that they believe to be current. When that view is stale — when an artifact is in flight but not yet acknowledged, when a tasking decision has been issued but its effects have not yet propagated, when both ends have written conflicting state during a link partition — the operator's decision is being made against assumptions that may not hold.

The custody problem is therefore not merely a transport-layer problem. It is a problem about what the operator sees, when, and with what staleness annotation. Any solution that does not surface freshness, custody state, and convergence status to the operator at decision time has not addressed the problem — it has only relocated it.

2.5 Multi-class operational examples

The §3 race-condition problem set is invariant across operational class. The following four classes illustrate the breadth; the unifying property — intermittent, low-bandwidth, high-latency, contended links between an edge-collection / edge-execution environment and a converging mission-state authority — holds across all of them.

2.5.1 Allied maritime forces in degraded-SATCOM regions. Surface vessels and submarines operate at sea under the authority of a shore-side mission-management center. For illustration, consider a fictitious operator — *Sovereign Patrol Authority* (SPA) — with a primary mission-management node at a continental shoreline base and forward stations at sub-tropical and sub-Antarctic relay points across a basin-scale operating area in the Southern Hemisphere. Vessels and the SPA shore center exchange traffic over a shared SATCOM channel; throughput is bounded by the ship's antenna aperture and the satellite's allocation, latency is multi-second, and the link drops under sea state, contention with other users in the spot beam, and adverse weather. The basin-scale operating area is large enough that forward operating relay points and polar-adjacent relay stations introduce additional intermittency on their own backhauls to the SPA shore center, compounding the custody challenge across multiple legs of the path. Mission-management decisions on shore — re-tasking, intelligence dissemination, threat alerting — must converge with the vessel's view of its own collected products and current operational state. The operator-decision loop runs at the SPA shore center, ingesting both fresh and stale information; without freshness markers and custody state, decisions are issued against assumptions that the in-flight state may have invalidated.

2.5.2 Military space command and control. Ground stations issue tasking to orbital assets through scheduled link windows; assets report state and return products through the same constrained channels. Multi-satellite constellations introduce multi-hop intermittency where a tasking decision may traverse two or three relay hops before reaching its destination, with each hop carrying its own custody-state implications. The operator-decision loop runs at the ground station and depends on a converged view of constellation state that is, by physical necessity, behind the leading edge of actual asset state.

2.5.3 Civil deep-space mission operations. Deep-space probes communicate with ground via DSN-class architectures over links with light-minute one-way latency and orbital-geometry-driven intermittent availability. The custody problem here is acute: artifacts may be in flight for hours; retransmit cycles span operational shifts; the ground mission-operations center must reason about in-flight state across a window where the naïve transport assumption (acknowledgement returns promptly) is violated by physics. The operator-decision loop runs at ground with explicit accommodation for the latency and intermittency; freshness markers and custody state are not optional accommodations but core protocol elements.

2.5.4 Expeditionary terrestrial operations. Disaster-response, austere-environment, and line-of-sight-denied operations all share the property of operating an edge command post that is intermittently connected to a converging operations center via whatever transport is available — frequently SATCOM, sometimes high-frequency radio, sometimes a chain of relays. The mission-management surface at the operations center must converge with the edge command post’s view of ground truth; the link characteristics share the four properties of §2.1 even though the physical regime differs from maritime or space.

Unifying claim. Across these four classes — and the broader class that includes commercial space constellation operators, autonomous mission systems with on-platform decision-making, disconnected AI inference at remote edges, and industrial edge regimes — the race-condition problem set of §3 is invariant. The capability envelope of §4 serves all of them. The orchestration pattern of §6 realizes the capabilities for all of them. The pattern is operationally neutral with respect to physical domain; what binds the class together is the structural property of the link, not the geography or the medium.

§3 The Race-Condition Problem Set

This section is the load-bearing problem-set anchor of the paper. We enumerate seven race conditions that arise in mission-state custody over intermittent links. For each, we specify the participants, the racing operations, the failure mode, the invariant that should hold but does not under naïve transport, and a forward reference to the capability in §4 that addresses it. Sequence diagrams are sketched in the figures cross-reference (figures 2.1 through 2.7) and described textually below where the rendering pass has not yet completed.

[OPERATOR-HANDOFF NEEDED — additional race classes from Sonota/Sonoto source notes may extend this set in v0.3+; v0.2 ships with seven as the operator-locked enumeration.]

3.1 R-1 — Bidirectional state-write race

Participants. Edge node and converging mission-state authority node, each holding a local replica of a logical mission-state object.

Racing operations. Both nodes write to the same logical state concurrently during a link partition or under conditions of mutual unawareness of in-flight messages.

Failure mode. When the partition heals, the system has two divergent states with no causal-ordering basis for reconciliation. Naïve wall-clock timestamps are unreliable due to multi-second clock drift between endpoints.

Invariant violated. *For any logical mission-state object, there shall exist a unique converged value derivable from a deterministic resolution rule over all writes against it.*

Why not solved by existing systems. Conflict-free replicated data types (CRDTs) provide a foundation for commutative resolution but do not natively encode domain-specific semantics for non-commutative operations such as tasking overrides. Battle-management systems typically assume a single authoritative writer with a well-connected link, which is precisely the assumption that fails in the degraded-link regime.

Capability that addresses this. C-2 (causally-ordered messaging) provides the ordering substrate; C-6 (bidirectional state convergence) applies the operator-specified resolution rules.

3.2 R-2 — Custody-vs-in-flight race

Participants. Edge node sending an artifact, converging authority receiving, and an operator querying custody state during the in-flight window.

Racing operations. Artifact A leaves the edge node at time t and arrives at the authority at time t' (where $t' - t$ may be seconds to minutes). An operator queries custody state at time t_q where $t < t_q < t'$; the authority's custody log shows A as missing or unknown.

Failure mode. Operator decision at t_q is made against the false belief that A does not exist. Operator may re-task collection of an artifact already produced, dismiss a query that should have been answered, or escalate a missing-artifact concern that is in fact resolved by physics-of-the-link.

Invariant violated. *For any artifact A with custody state s, the operator's view of s shall accurately reflect that A is in-flight rather than missing, when in-flight is the true state.*

Why not solved by existing systems. WAN acceleration appliances optimize throughput but do not surface artifact-level in-flight state to a mission-management UI. UDP file-transfer protocols provide per-transfer progress but lack a globally consistent artifact-identity discipline across heterogeneous workloads.

Capability that addresses this. C-1 (custody-aware transport) carries the artifact identity and state-machine; the operator UI surfaces the in-flight state.

3.3 R-3 — Freshness race

Participants. Both endpoints, each holding a cached copy of the same logical object, each having updated that copy at some time after the last mutual synchronization.

Racing operations. Both endpoints have a “more recent” version by their own local accounting; without a causally consistent ordering, neither knows whose version supersedes.

Failure mode. Whichever side’s version reaches the other first is taken as authoritative, leading to a silent loss of the slower-arriving update. Naïve wall-clock timestamps fail because multi-second clock drift between endpoints means timestamps are not reliably orderable.

Invariant violated. *For any pair of mutations m , m' to the same logical object, the system shall determine an unambiguous ordering relation between them and apply both in that order.*

Why not solved by existing systems. Eventual-consistency databases address this at the data layer but typically do not propagate freshness annotation through to the operator-facing UI; the operator sees a value with no indication of how stale it may be relative to an alternate view.

Capability that addresses this. C-3 (freshness markers) makes staleness explicit at the UI; C-2 (causally-ordered messaging) provides the ordering basis.

3.4 R-4 — Operator-decision race

Participants. Operator at the converging authority, edge-side process generating fresher state, and the link.

Racing operations. Operator queries mission state at t_q and acts on it, issuing a tasking or instruction T at $t_q + \delta$. Meanwhile, a fresher state update U is in flight from edge, having left at $t < t_q$. U lands at $t_q + \delta$ where δ may be greater or less than δ .

Failure mode. Operator’s tasking T is issued under assumptions that U invalidates. The tasking propagates back to the edge, where it is acted on against a state that has already moved past the assumptions T relied on.

Invariant violated. *For any operator decision T issued at time $t_q + \delta$, the operator shall be informed at decision time of the freshness of the state T is based on, and shall be alerted if a fresher update U is known to be in flight.*

Why not solved by existing systems. Standard mission-management portals present state without freshness annotation and without in-flight-update awareness. The operator is left to compute staleness by inference.

Capability that addresses this. C-3 (freshness markers) annotates the operator’s view; C-8 (operator-bounded autonomy) ensures the orchestration layer can warn or pause for operator confirmation when fresh state is known to be in flight.

3.5 R-5 — Re-request race

Participants. Edge node, converging authority node, and the link, with each side independently noticing that an artifact appears to be missing.

Racing operations. Edge notices that artifact A has not been acknowledged by the authority within an expected window and initiates a re-send. Simultaneously, authority notices that A is expected but not received and initiates a re-request. Both messages cross on the link.

Failure mode. Artifact A is transmitted at least twice; receipt is ambiguous (which copy was acknowledged?); custody log accumulates duplicate entries; the authority may store both copies or arbitrarily discard one.

Invariant violated. *For any artifact A, re-request and re-send operations shall be idempotent and globally serializable, such that exactly one canonical copy exists at the authority and the custody log reflects this.*

Why not solved by existing systems. Transport-layer retransmit (TCP, QUIC [RFC 9000]) addresses packet-level idempotency but not artifact-level idempotency. File-transfer protocols address whole-file idempotency but not the operator-visible custody log.

Capability that addresses this. C-1 (custody-aware transport) provides hash-verified identity and idempotent operations; C-5 (atomic resume semantics) provides reconnection-time disambiguation.

3.6 R-6 — Reconnect race

Participants. Both endpoints, each having locally maintained a believed-state during a link drop.

Racing operations. Link drops at t_d , both sides continue local work and form independent beliefs about what was in flight and what was acknowledged. Link recovers at t_r ; both sides attempt to resume transfer from their local beliefs without first synchronizing on a globally consistent resume point.

Failure mode. Transfers resume from inconsistent positions; partial artifacts are re-sent in their entirety or skipped; the in-flight queue diverges from the receive queue; debugging is intractable because there is no record of the resume-point disagreement.

Invariant violated. *Upon link recovery, both endpoints shall agree on a single resume point determined by a handshake protocol before any application-layer traffic flows.*

Why not solved by existing systems. WAN acceleration and UDP file-transfer protocols address per-flow resume but not artifact-level resume with operator-visible state. Delay-tolerant networking (BPv7, RFC 9171) addresses bundle-level resume and is the closest prior art at this point; see §5.

Capability that addresses this. C-5 (atomic resume semantics).

3.7 R-7 — Priority-inversion race

Participants. A high-priority command-and-control message and a large low-priority sensor-of-interest binary, both being transmitted across the link in fragmented form.

Racing operations. The low-priority binary's chunks are mid-stream when the high-priority C2 message arrives at the sender's queue. The fragmentation discipline determines whether the C2 message must wait for the binary's chunks to clear (head-of-line blocking) or whether C2 chunks can interleave with the binary's chunks.

Failure mode. Under head-of-line blocking, the C2 message experiences latency proportional to the unconsumed remainder of the binary; on a low-bandwidth link, this latency may violate operational requirements (a tasking change arriving after the window in which it would have been actionable).

Invariant violated. *For any message of priority class p , the latency from enqueue to receipt shall not exceed a bound determined by the priority class, independent of the size of lower-priority messages already in flight.*

Why not solved by existing systems. QoS-based packet prioritization addresses this at the packet layer but does not extend cleanly to artifact-level priority across application-layer fragmentation. Standard L4 protocols cannot interleave application-layer artifacts mid-flight without breaking the artifact’s reassembly invariants.

Capability that addresses this. C-4 (priority-aware fragmentation) preserves priority across chunking and supports interleaving.

3.8 Summary

The seven race conditions enumerated above constitute a problem class — a structural feature of mission-state custody over intermittent links — that any candidate solution must address in full. In §4 we specify the complementary capability envelope; in §5 we show that no existing product or protocol addresses the full envelope; in §6 we present one realization.

§4 Capabilities Required

This section is the load-bearing requirements anchor. We specify ten capabilities (C-1 through C-10) that any solution to the §3 problem set must realize. The capabilities are *complementary by construction* — each addresses a distinct facet of the problem set; no two capabilities overlap; together they form a closed envelope such that removing any one leaves part of §3 unaddressed.

Each capability is presented as a *requirement*, not an implementation. §6 presents one realization; alternative realizations are possible and the requirements stated here do not prejudge them.

4.1 C-1 — Custody-Aware Transport

What it does. Every artifact carries a globally consistent identity (content-hash plus sequence number) and a state-machine: created → enqueued → in-flight → received → acknowledged → durable. Both sides maintain a hash-verified inventory and can query each other’s view of any artifact’s state.

Race conditions addressed. R-2 (custody-vs-in-flight) — primary; the in-flight state is explicitly representable. R-5 (re-request) — primary; the hash-verified identity makes re-send and re-request idempotent.

Complementarity note. C-1 is the substrate for the rest of the envelope. C-3 (freshness markers) attaches age metadata to a C-1 artifact handle; C-5 (atomic resume) depends on C-1 to know what was in flight pre-drop; C-10 (defensive replay-ability) records C-1 state transitions in the audit log.

Verifiability hook. A §7 validation harness can inject an artifact, query custody state at random intervals during the in-flight window, and verify the response correctly reflects the artifact’s lifecycle position.

4.2 C-2 — Causally-Ordered Messaging

What it does. Message ordering is driven by causal markers — vector clocks, logical clocks, or operator-injected causal identifiers — rather than wall-clock timestamps. The system tolerates multi-second clock drift between endpoints.

Race conditions addressed. R-1 (bidirectional state-write) — primary; causal ordering provides the basis for resolution. R-4 (operator-decision) — contributory; the operator’s tasking can be causally ordered against in-flight state updates.

Complementarity note. C-2 is orthogonal to C-6: C-2 specifies how events are ordered; C-6 specifies how ordered events resolve into final state when they conflict semantically.

Verifiability hook. Inject divergent writes across a simulated partition; verify that the resolved ordering is consistent regardless of which side’s wall-clock is leading or lagging.

4.3 C-3 — Freshness Markers

What it does. Every cached object self-reports its age relative to a known authoritative source. Markers are visible to both the orchestration layer and the operator-facing UI; staleness propagates through downstream caches without requiring re-computation.

Race conditions addressed. R-3 (freshness race) — primary. R-4 (operator-decision) — primary; the operator sees staleness at decision time.

Complementarity note. C-3 builds on C-1’s stable artifact handle and on C-2’s causal ordering; it does not duplicate either, but attaches operator-relevant age metadata at the UI surface.

Verifiability hook. Cache a value; cause a divergent update; query the cached value’s freshness marker; verify that the marker correctly indicates staleness relative to the authoritative source.

4.4 C-4 — Priority-Aware Fragmentation

What it does. Fragmentation discipline preserves message-level priority across chunking, supporting interleaving without head-of-line blocking. High-priority command-and-control chunks can pass low-priority sensor-of-interest binary chunks mid-stream.

Race conditions addressed. R-7 (priority-inversion) — primary.

Complementarity note. C-4 collaborates with C-7 (adaptive bandwidth shaping). C-4 decides priority order; C-7 decides how much bandwidth each priority class receives at the current link state.

Verifiability hook. Enqueue a large low-priority binary; mid-transfer enqueue a small high-priority message; measure the high-priority message’s latency under both head-of-line-blocking discipline and priority-aware-fragmentation discipline; verify the latter falls within the priority class’s latency bound.

4.5 C-5 — Atomic Resume Semantics

What it does. When the link drops and recovers, both sides resume from a known-consistent state determined by a reconnection handshake, not from independently-held local beliefs.

Race conditions addressed. R-6 (reconnect race) — primary.

Complementarity note. Depends on C-1 (custody state) to know what was in flight pre-drop; collaborates with C-2 (causal ordering) to merge state safely on resume.

Verifiability hook. Drop the link mid-transfer at random points; recover; verify both sides agree on the resume point before any application-layer traffic resumes.

4.6 C-6 — Bidirectional State Convergence

What it does. When both sides write to the same logical mission state during link partition, the orchestration layer applies operator-specified resolution rules — CRDT-style for commutative operations, domain-specific resolvers (e.g., tasking-override semantics) for non-commutative operations. Resolution rules are declarative and operator-auditable.

Race conditions addressed. R-1 (bidirectional state-write) — primary, in conjunction with C-2.

Complementarity note. C-2 orders events; C-6 specifies what to do when ordered events conflict semantically. The two are necessary and sufficient together for resolving R-1; neither alone suffices.

Verifiability hook. Inject conflicting writes during a partition; recover; verify the resolved state matches the operator-specified rule's expected outcome.

4.7 C-7 — Adaptive Bandwidth Shaping

What it does. The orchestration layer senses instantaneous link state (effective bandwidth, jitter, loss) and adjusts chunk sizes, retransmit cadence, and priority-class budgets accordingly.

Race conditions addressed. No §3 race directly; required for C-4 and C-5 to function under varying link conditions.

Complementarity note. Provides the dynamic-context input that prioritization (C-4) and resume (C-5) key off. Without C-7, C-4's priority decisions would be made against stale bandwidth assumptions.

Verifiability hook. Vary the simulated link characteristics over time; verify the orchestration's chunk sizes, retransmit cadence, and priority-class budgets adjust in response.

4.8 C-8 — Operator-Bounded Autonomy

What it does. Autonomous components execute deterministically within an envelope the operator defines and can revoke at any time — token revocation, capability revocation, persona pause. The operator can see exactly what the autonomous layer is doing and why.

Race conditions addressed. Governance shadow of R-4 (operator-decision race) — ensures the operator remains the decisive node even when the autonomous substrate is in motion.

Complementarity note. C-8 is the governance layer wrapping all other capabilities. Without it, the custody guarantees of C-1 through C-7 become unauditable autonomy. C-8 is what makes the rest of the envelope a tool the operator wields rather than a substitute for the operator.

Verifiability hook. Revoke a persona’s execution token mid-operation; verify the persona halts deterministically and the operator UI surfaces the revocation event and the persona’s last-completed action.

4.9 C-9 — Cross-Link Triangulated Diagnostics

What it does. Fault attribution across both sides of the link, fused from co-temporal observability data. When the link degrades or an artifact fails to arrive, both endpoints contribute evidence and the orchestration layer attributes fault to one of {edge node, link, converging authority node, operator}.

Race conditions addressed. Operational shadow of R-2, R-5, R-6 — makes failure modes attributable rather than ambiguous.

Complementarity note. Provides the closed-loop signal that C-7 needs to adapt bandwidth and that C-8 needs to make governance decisions on degraded paths.

Verifiability hook. Inject faults at known locations (edge, link, authority); verify the diagnostic attribution matches the injected fault location to within an acceptable accuracy bound.

4.10 C-10 — Defensive Replay-Ability

What it does. Every persona action and every state transition is durably logged in a tamper-evident form that can be replayed deterministically for post-hoc audit, forensics, or operator review.

Race conditions addressed. No §3 race directly; provides the trust substrate that makes C-6’s resolution rules, C-8’s governance envelope, and C-9’s attribution claims credible.

Complementarity note. C-10 is the audit floor under all other capabilities. Required for any classified or contested deployment context.

Verifiability hook. Run a session; capture the replay log; replay deterministically; verify the replayed sequence is bit-identical to the original sequence and that any divergence is itself a flagged event.

4.11 The complementarity matrix

The ten capabilities are complementary by construction. We document the coverage explicitly:

Capability	R-1 state-write	R-2 custody- flight	R-3 freshness	R-4 operator- decision	R-5 re-request	R-6 reconnect	R-7 priority- inversion
C-1 custody- aware transport	—	primary	—	—	primary	contributory	—
C-2 causal ordering	primary	—	contributory	contributory	—	contributory	—
C-3 freshness markers	—	—	primary	primary	—	—	—
C-4 priority fragmentation	—	—	—	—	—	—	primary
C-5 atomic resume	—	—	—	—	contributory	primary	—
C-6 state convergence	primary	—	—	contributory	—	contributory	—
C-7 adaptive shaping	—	—	—	—	—	contributory	contributory
C-8 operator- bounded autonomy	contributory	—	—	primary	—	—	—
C-9 cross-link diagnostics	—	contributory	—	—	contributory	contributory	—
C-10 defensive replay	contributory	contributory	contributory	contributory	contributory	contributory	contributory

Observations on the matrix:

- **No race condition R-N is empty in its column.** Every race has at least one capability marked “primary” against it. The envelope is *complete* with respect to the §3 problem set.
- **No capability C-M is empty in its row.** Every capability contributes to at least one race condition. The envelope is *minimal* in the sense that every capability has a job.
- **C-10 is contributory across the entire row.** This reflects its role as the audit floor: every other

capability’s claim is made credible by the replay log.

- **C-8 is primary on R-4 and contributory on R-1.** This reflects its role as the governance layer: the operator-decision race has a governance shadow that C-8 specifically addresses (ensuring the operator sees freshness at decision time and remains the decisive node).

4.12 Why this is novel as a set

Individual capabilities have prior art. Custody hashes appear in version-control systems; vector clocks appear in distributed databases; priority queueing appears in QoS networking; CRDTs appear in eventual-consistency systems. The contribution of this paper is *the integration of these capabilities as a complementary envelope above Layer 7, specifically for mission-state custody across intermittent links*. No prior published reference architecture composes them this way for this problem class.

The closest prior compositions (CRDT databases composed with WAN acceleration; battle-management systems composed with delay-tolerant networking transport) address parts of the envelope but leave specific facets uncovered — particularly the operator-facing freshness annotation (C-3), the operator-bounded autonomy (C-8), and the defensive replay-ability (C-10) as a composed whole. §5 documents the gaps in detail.

The capabilities of §4 are *requirements*: what any solution must do, regardless of how. §6 presents one realization, and the proto-specification extensions of §6.10–§6.13 extend that realization with the doctrine for persona contracts, custody object model, governance arbitration, and freshness semantics that the envelope’s behavioral guarantees require.

§5 Prior Art and Where It Falls Short

This section surveys the prior-art landscape across categories. For each category and each prior-art line, we identify the §4 capabilities it addresses (in whole or in part) and the residual gap relative to the §3 race-condition problem set. The intent is fair characterization — these technologies solve adjacent problems competently, and the gap lives at the orchestration layer above them. Specific representative products and protocols within each category are consolidated in **Appendix C — Representative Prior-Art Technologies**.

5.1 WAN acceleration

WAN acceleration appliances optimize throughput across high-latency or lossy links via techniques including data deduplication, compression, protocol acceleration, and connection pooling. Representative products in this category are listed in Appendix C.

These products address effective bandwidth on a degraded link and contribute to C-7 (adaptive bandwidth shaping) in that they perform link-state-aware adjustment at the transport layer. They do not provide artifact-level custody (C-1), causal ordering (C-2), operator-facing freshness markers (C-3), or any of the governance and replay capabilities (C-8, C-10). They are necessary but not sufficient infrastructure beneath an orchestration layer that does provide those.

5.2 Delay-Tolerant Networking family — load-bearing prior art

Delay-tolerant networking (DTN) was developed for deep-space and intermittent-link communications by the Jet Propulsion Laboratory and the Internet Research Task Force DTN Research Group. The relevant standards are:

- **Bundle Protocol Version 7** [2, RFC 9171] — the current standard for bundle-layer transfer over delay-tolerant links, supporting custody transfer, fragmentation, and store-and-forward operation.
- **Licklider Transmission Protocol** [3, RFC 5326] — the convergence-layer protocol for bundle transfer over deep-space links with very long round-trip times.
- **Delay-Tolerant Networking Architecture** [4, RFC 4838] — the architectural foundation describing the bundle-layer overlay above conventional transport.
- **CCSDS Bundle Protocol Specification** [5, CCSDS 734.2-B-1] — the Consultative Committee for Space Data Systems' Blue Book specification incorporating BPv7 for space mission use.
- **DTN academic record** — Burleigh et al., *Delay-Tolerant Networking: An Approach to Interplanetary Internet*, IEEE Communications Magazine 41(6), 2003 [UNVERIFIED — editorial confirms volume/issue].

DTN is the closest existing prior art to the problem set of this paper. Its mapping to the capability envelope of §4:

- **C-1 (custody-aware transport) — primary realization at transport layer.** DTN's bundle custody transfer mechanism is conceptually equivalent at the protocol layer to the custody discipline of C-1. Bundles carry a globally consistent identity; custody transfers are explicit; the bundle layer maintains a custody state per bundle.
- **C-5 (atomic resume semantics) — partial realization.** DTN's fragmented bundle reassembly provides resume capability at the bundle layer, supporting recovery from link drops within a transfer.

DTN does *not* provide:

- **C-3 (freshness markers visible to operator).** DTN operates below the application layer and does not natively surface staleness annotation to a mission-management UI.
- **C-6 (bidirectional state convergence with operator semantics).** DTN provides transport-layer custody; it does not specify what to do when two endpoints have written conflicting state to the same logical mission object during a partition. The semantic resolution layer is application-level.
- **C-8 (operator-bounded autonomy).** DTN is a transport-layer protocol; governance of autonomous decision-making above the transport layer is out of scope.
- **C-10 (defensive replay-ability) composed with C-8.** DTN's protocol logs are operationally useful for protocol debugging; they are not the tamper-evident operator-replay log that C-10 specifies.

Synthesis claim. DTN solves transport-layer custody for deep-space-class problems and is the foundation that an above-L7 orchestration layer for mission-state custody should compose with rather than replicate. The orchestration layer this paper proposes operates above DTN-style transport, composing transport-layer custody with operator-curated mission-state semantics. The integration is the contribution — not transport-layer custody alone.

5.3 UDP file transfer and high-loss-tolerant transport

This category includes proprietary high-speed file-transfer protocols, open-source resilient-UDP transports, and modern multiplexed transports. Representative products and standards are listed in Appendix C. QUIC [RFC 9000] is the most relevant standardized transport in the category.

These protocols address efficient bulk transfer over high-loss, high-latency links. They contribute partially to C-1 (custody-aware transport) at the per-transfer level — whole-file hash verification, idempotent re-transfer — but lack a globally consistent artifact-identity discipline across heterogeneous workloads, lack operator-facing freshness annotation, and lack the orchestration-level state machine that C-1 specifies.

QUIC specifically [RFC 9000] addresses connection-level reliability, multiplexing, and modern transport security. It is an excellent foundation but operates at L4; the orchestration layer of this paper assumes a transport like QUIC or DTN below it.

5.4 Conflict-free replicated data types and eventual-consistency systems

Representative academic work and production systems are listed in Appendix C, including foundational CRDT formal-systems work [6] and production CRDT and eventual-consistency databases.

These provide the formal foundations and production realizations for commutative state convergence. They address part of C-6 (bidirectional state convergence) — specifically the commutative-operations case. They do not natively encode domain-specific semantics for non-commutative operations such as tasking overrides, and they do not propagate freshness annotation through to operator-facing UI surfaces.

5.5 Mission-management and battle-management systems

This category includes the operator-facing portals and command-and-control suites that present mission state to operators. Representative products are listed in Appendix C, including products developed for allied military and adjacent national-system contexts.

These systems address the operator-facing mission-management surface. They support the operator's decision loop on a converged view of mission state. They typically assume a well-connected operations center and do not natively address the §3 race conditions that arise in degraded-link operations. The orchestration layer of this paper would compose beneath such a battle-management system, providing the custody and convergence guarantees that the BMS assumes but cannot itself enforce under degraded-link conditions.

5.6 Operations monitoring

The observability category includes the agent-and-server monitoring platforms that collect co-temporal telemetry from edge and shore. Representative products are listed in Appendix C.

These provide the observability substrate. They contribute to C-9 (cross-link triangulated diagnostics) in that they collect the co-temporal evidence from edge and converging authority that fault attribution depends on. They do not provide the fusion logic that triangulates fault across the link — that fusion is the orchestration-layer contribution.

5.7 Edge transport firewalls and hardened compute

This category includes ruggedized perimeter-security appliances and hardened compute platforms intended for shipboard, airborne, or expeditionary deployment. Representative products are listed in Appendix C.

These provide the security and ruggedized-compute substrate. They are infrastructure beneath the orchestration layer; they do not address the orchestration concerns directly.

5.8 Edge AI inference

This category includes platform-level inference compute, runtime libraries, and model-server infrastructure deployable at the edge. Representative products are listed in Appendix C.

These produce edge artifacts that the orchestration layer must move across the link with custody intact. They are upstream producers of artifacts that the orchestration discipline of §6 manages.

5.9 LEO constellation backhaul

This category includes the satellite-backhaul services that supply intermittent connectivity to maritime, airborne, and expeditionary edge nodes. Representative offerings are listed in Appendix C. Recent academic measurement work on LEO constellation performance [UNVERIFIED — LEO measurement papers 2023-2025] characterizes the link properties this paper assumes.

These are the transport substrate at the link layer. They provide the link characteristics that C-7 (adaptive bandwidth shaping) keys off; they do not address the orchestration concerns.

5.10 Synthesis: the structural gap

No single product or protocol surveyed above addresses the full §3 race-condition problem set. Each addresses part. The gap is structural and lives at the orchestration layer above Layer 7:

- The transport-layer products (WAN acceleration, DTN, UDP file transfer, QUIC) cannot carry mission semantics.
- The application-layer products (BMS, mission-management portals) cannot observe transport state.
- The data-layer products (CRDT databases, eventual-consistency systems) cannot natively encode domain-specific resolution semantics or propagate freshness to the operator surface.
- The observability products provide raw evidence but not fused attribution.

The orchestration layer of §6 occupies the structural gap.

§6 The Orchestration Pattern

This section presents one realization of the capability envelope of §4. The realization is presented at conceptual specificity — sufficient to establish the contribution and the design intent, not sufficient to compromise implementation specifics. Each element is mapped explicitly back to the capability or capabilities it realizes, so the reader can trace the chain from problem (§3) through requirement (§4) to realization (§6).

6.0 Security and threat-model framing

The orchestration substrate this paper describes composes with — and does not replace — standard transport security (TLS, IPsec), link-layer encryption, and authentication / authorization mechanisms. The substrate sits above L7 and operates on already-secured channels; threats to the underlying transport are the responsibility of the link / network / transport layers. The substrate’s threat model is concerned with mission-state integrity under intermittent connectivity and operator-bounded autonomy enforcement, not with confidentiality of the transport itself. Where this paper speaks of operator-revocable tokens, persona contracts, custody chains, and replay logs, those mechanisms presuppose a securely-authenticated transport beneath; they do not provide that security themselves.

6.1 Placement: above Layer 7

The orchestration pattern is placed above the application layer in the OSI stack. The placement is motivated by §5’s structural-gap analysis: transport-layer products cannot carry mission semantics, application-layer products cannot observe transport state, and the gap that the §3 problem set occupies is structurally above L7.

The above-L7 placement composes with transport-layer protocols beneath it (DTN, QUIC, WAN acceleration appliances) and with application-layer surfaces above it (mission-management portals, decision-support tools). The pattern is transport-agnostic — it does not require any particular L4 protocol, though it benefits from a transport that provides custody primitives such as DTN.

6.2 Persona decomposition

The orchestration pattern decomposes its responsibilities into three deterministic agent personas, each with a formally specified interface contract and a revocable execution token. The decomposition is the key novelty claim: structured persona decomposition (rather than monolithic agents or monolithic procedural code) is used to achieve custody guarantees under operator governance.

Forwarder. Realizes C-4 (priority-aware fragmentation) and contributes to C-7 (adaptive bandwidth shaping). The Forwarder specifies QoS rules for traffic crossing the link, governs the chunk-sizing strategy as a function of current link state, and enforces the interleaving discipline that prevents priority-inversion of high-priority C2 messages behind large low-priority binaries.

Custody Manager. Realizes C-1 (custody-aware transport) and C-2 (causally-ordered messaging) at the artifact level. The Custody Manager specifies the hash-inventory protocol, maintains the artifact state machine, governs idempotent re-request logic, and stamps causal-ordering markers on outgoing messages.

Fragmentation Manager. Collaborates with the Forwarder on C-4. The Fragmentation Manager owns reassembly ordering on the receive side and contributes to C-5 (atomic resume semantics) by tracking partial-receipt state across link drops and recoveries.

The three personas execute deterministically — bounded, observable, replayable — and are not full large-language-model agents making free-form decisions. The persona decomposition realizes C-8 (operator-bounded autonomy) by construction: each persona’s contract is well-specified; each persona’s execution

token is revocable by the operator at any time; each persona's actions are logged to the C-10 replay log. The proto-specification of persona contracts is detailed in §6.10.

6.3 State convergence layer

The state convergence layer sits alongside the Custody Manager and realizes C-6 (bidirectional state convergence). It applies operator-specified resolution rules — CRDT-style for commutative operations, domain-specific resolvers for non-commutative operations such as tasking-override semantics. The resolution rules are declarative — they are written by the operator in a rule language that is auditable, version-controlled, and replayable.

The state convergence layer's input is a set of writes that have been causally ordered by the Custody Manager (via C-2). Its output is a single converged state per logical object, computed deterministically from the input writes and the operator-specified rule set. Every convergence decision is logged to the C-10 replay log with its inputs and the rule that produced it. When the convergence layer cannot deterministically resolve a conflict within the operator-specified rule envelope, the system enters arbitration mode per §6.12.

6.4 Freshness telemetry

The freshness telemetry layer realizes C-3 (freshness markers). Every cached object carries a timestamped staleness annotation that is computed at cache-write time and propagated through downstream caches without re-computation. The freshness annotation is visible to both the orchestration layer's internal decision-making and to the operator-facing UI.

The annotation is not a wall-clock timestamp (which would be vulnerable to clock drift) but a causal-ordering position relative to the authoritative source's most recent update. The operator UI surfaces this position as a human-readable staleness indicator (e.g., “this view is N causal steps behind authoritative”). The behavioral guarantees that freshness telemetry provides for operator-decision support are formalized in §6.13.

6.5 Resume protocol

The resume protocol realizes C-5 (atomic resume semantics). On link recovery, both endpoints exchange a state-vector handshake before any application-layer traffic resumes. The handshake encodes each side's view of the in-flight queue, the receive queue, and the last-acknowledged position. The orchestration determines the consistent resume point from the handshake — typically the most recent position that both sides agree on — and resumes from there.

If the handshake reveals an inconsistency that cannot be safely resolved (e.g., divergent state without operator-specified resolution rules), the orchestration halts and surfaces the divergence to the operator via C-8's operator-decision pathway, which is formalized in §6.12 as governance-state arbitration.

6.6 Bandwidth-state sensor

The bandwidth-state sensor realizes C-7 (adaptive bandwidth shaping). It continuously estimates link state — effective bandwidth, jitter, loss — from observed protocol-layer evidence and feeds the estimate to the

Forwarder and Fragmentation Manager. The Forwarder adjusts priority-class budgets accordingly; the Fragmentation Manager adjusts chunk sizes.

The sensor's estimate is also fed to C-9 (cross-link triangulated diagnostics) as an input to fault attribution.

6.7 HGC³AE² governance overlay

The orchestration is governed by the HGC³AE² framework — Human Governance and Curation of Adaptive Autonomous Agentic Entities. The governance overlay realizes C-8 (operator-bounded autonomy) and C-10 (defensive replay-ability).

HGC³AE² specifies:

- **Revocable execution tokens.** Every persona executes under a token issued by the operator. The token specifies the persona's capability scope (what it may do), its temporal scope (when it may do it), and its data scope (what artifacts it may access). The operator can revoke any token at any time; revocation halts the persona deterministically.
- **Operator-visible decision surface.** Every autonomous decision is annotated with its inputs, the rule that produced it, and the persona that made it. The operator UI surfaces this annotation alongside the decision's effect, so the operator can audit any decision in real time or post hoc.
- **Defensive replay log.** Every persona action and every state transition is durably logged in a tamper-evident form. The log supports deterministic replay for forensics, audit, or operator review. Replay is bit-identical to the original sequence (within the bounds of deterministic-replay guarantees of the underlying compute substrate).

HGC³AE² is the governance frame that makes the rest of the orchestration trustworthy in classified or contested deployment contexts. The contract-level specifics of how personas accept inputs, produce outputs, and submit to revocation are formalized in §6.10.

6.8 Cross-link triangulated diagnostics

The cross-link diagnostics layer realizes C-9 (cross-link triangulated diagnostics). Edge-side and converging-authority-side observability data are fused into a single fault-attribution surface. Observability is collected via standard tooling (per the category-level discussion in §5.6 and Appendix C); the orchestration's contribution is the fusion logic that triangulates fault across the link, attributing each failure to one of {edge, link, authority, operator} with a confidence estimate.

The attribution output feeds back into C-7 (adaptive bandwidth shaping) — if the link is the attributed fault, bandwidth shaping responds — and into C-8 (operator-bounded autonomy) — if the operator is the attributed fault, governance responds with appropriate alerting.

6.9 Replay log

The replay log realizes C-10 (defensive replay-ability). Every persona action and every state transition is appended to a tamper-evident log. The log is structured for deterministic replay — given the log and the initial

state, the replay reproduces the original sequence bit-identically (modulo non-determinism in the compute substrate, which is itself logged).

The log is the trust substrate that makes C-6's resolution rules, C-8's governance envelope, and C-9's attribution claims credible. It is required for any classified or contested deployment context.

6.10 Deterministic Persona Contracts

The persona decomposition of §6.2 is grounded in a contract-level discipline that determines what a persona may do, how it does it, and how its actions compose into operator-auditable mission-state histories. This subsection formalizes that discipline at the semantic level. Implementation specifics — including the data structures that carry contract metadata, the algorithms that evaluate contracts, and the wire format for tokens — are matters of engineering specification outside this paper's scope.

Bounded authority surfaces. Each persona is defined by a contract that enumerates exhaustively: the kinds of state the persona may read, the kinds of state it may write, and the operations it may trigger. Outside-contract actions are inadmissible by construction at the orchestration substrate level — the substrate refuses to admit an action that falls outside the persona's contract envelope, rather than admitting it and relying on post-hoc audit to flag the violation. This *admission-time refusal* is what distinguishes the persona model from a permissive-then-audit model.

Persona contract model. A persona contract specifies four faces of the persona's interaction with the substrate:

- *Inputs* — typed, with each input bound to a custody-marked predecessor. The contract enumerates the input categories the persona accepts and the type constraints each category must satisfy.
- *Outputs* — typed and custody-marked. The contract enumerates what the persona may produce, what custody chain each output inherits from its inputs, and what downstream personas may consume each output category.
- *Preconditions* — the substrate state that must hold for the persona to admit an action. Preconditions include: operator-bounded-autonomy envelope is active for this persona; the persona's execution token is valid and unrevoked; the input artifacts satisfy the contract's input categories and custody requirements.
- *Postconditions and effects* — the state-machine transitions the persona's action produces, and which downstream personas observe the effects. Contracts are declarative; the substrate enforces postconditions deterministically.

Contracts are operator-auditable: each contract version is durably recorded; the operator can examine any persona's contract at any time; contract changes are themselves governance events that the substrate records.

Deterministic execution guarantees. Given identical inputs (including identical custody state and identical causal markers), a persona produces identical outputs and identical downstream effects. Determinism is a contract-level property, not an implementation-level property: machine-learning or large-language-model components MAY exist internally to a persona, but they MUST be wrapped in deterministic admission semantics — the persona accepts an ML output only when that output passes deterministic admission

rules (typed, in-contract, custody-marked, within operator-bounded confidence thresholds). The persona's externally-observable behavior is determined by those admission rules, not by the stochasticity of any internal component. This composition — deterministic envelope over potentially-stochastic internals — is what permits operator-trustworthy use of contemporary AI components within the substrate.

Replay semantics. Any persona action is replayable bit-exact given the recorded inputs and the persona's contract version. Replay is the audit instrument; replay-divergence is a defect that the substrate surfaces as a flagged event. The replay log of §6.9 binds together the persona contract version, the inputs, the outputs, the custody markers, the token under which the action ran, and the governance state at the time of the action — together these constitute the audit substrate that gives the operator durable visibility into autonomous behavior.

Operator-visible action chains. Each persona action emits an operator-observable record carrying: the persona identity and contract version; the inputs (with their custody markers); the outputs (with their assigned custody markers); the token under which the action ran; the governance state at action-admission time. Action chains compose into mission-state histories — sequences of persona actions linked by custody lineage — that the operator can inspect at any depth.

Token-scoped permissions. Every persona action carries an operator-issued token that scopes the action's authority. Tokens carry, at the semantic level: a *scope* (what the token authorizes — capability, temporal window, data domain); an *expiry* (when the token ceases to be valid); a *revocability metadata* (the operator authority that may revoke the token and the conditions under which revocation propagates downstream). The exact data structure of a token is an implementation matter and not specified here; the *fact* of token-scoped permission and the *behavior* of the substrate under token-scoped actions is the doctrine.

Revocation behavior. Operator revocation of a token terminates any in-flight action under that token at the next safe-point of the persona contract. Safe-points are points at which the persona's state machine permits cessation without leaving inconsistent custody state — they are part of the contract specification. Revocation propagates through downstream personas via custody-chain dependencies: when a token is revoked, any action that consumed an output produced under that token enters a custody-pending state pending operator arbitration per §6.12. Revocation is not advisory; the substrate enforces it. This is the load-bearing property that makes operator control meaningful — the operator's revocation authority is not a request the substrate considers but a halt the substrate executes.

6.11 Custody Object Model

The custody discipline of §6.2 is grounded in a conceptual structure for custody objects that the substrate maintains for every artifact under custody. This subsection formalizes that structure at the semantic level — defining what custody objects carry, how they transition through their lifecycle, and how custody handoff and rollback are governed. Implementation specifics — including the on-the-wire encoding, the hash algorithms used for content addressing, the storage format of the custody log, and the wire format of the replay-log binding — are matters of engineering specification outside this paper's scope.

Conceptual fields. A custody object carries the following conceptual fields, each defined by its purpose and behavioral role rather than by its data type or encoding:

- *Artifact ID* — a globally consistent identifier for the artifact. Uniqueness is invariant: no two distinct artifacts share the same ID; one artifact has exactly one ID throughout its lifecycle.
- *Causal sequence ID* — a value that orders this artifact’s lifecycle events relative to other artifacts within the same mission scope. The causal sequence is not a wall-clock timestamp; it is a causal-ordering position that supports the §6.13 freshness semantics.
- *Lineage* — the chain of derivations from predecessor artifacts. Lineage records, at the semantic level, which artifacts this one was produced from and which persona contract produced the derivation. Lineage is append-only within a custody object.
- *Freshness vector* — a multi-dimensional staleness indicator. The vector carries, at the semantic level: recency relative to origin (when did the underlying observation occur?); recency relative to the operator-decision timeline (when did the operator last act on this artifact?); recency relative to the mission timebase (where in the mission’s logical time does this artifact sit?). The freshness vector’s behavioral role is detailed in §6.13.
- *Authority provenance* — which operator-issued token, which persona contract, and which governance state produced this artifact. Authority provenance is what makes the artifact’s existence traceable to a specific operator-bounded-autonomy authorization.
- *Custody lifecycle state* — the artifact’s current state in the lifecycle (enumerated below).
- *Replay binding* — a pointer (conceptual; not a specific implementation reference) into the deterministic replay log, such that the artifact’s full lifecycle history can be regenerated from the replay log and the artifact’s lineage. The replay binding is the audit-time recoverability anchor.
- *Convergence metadata* — markers used during state convergence (§6.3, §6.12) to resolve write conflicts. Convergence metadata records, at the semantic level, the resolution rule applied to derive this artifact’s current state from competing predecessor versions.

Expanded lifecycle states. The custody object’s lifecycle state is one of eight:

1. **Created** — the artifact has been materialized at edge or shore, but it has not yet been committed to custody. In this state, the artifact exists but is not yet part of the mission custody chain.
2. **Staged** — the artifact has been committed to local custody and is ready to enter the forwarding queue. Custody hashing and lineage assignment have completed; the artifact is durable locally.
3. **Queued** — the artifact has been accepted into the forwarding queue with a priority class assigned. The Forwarder (§6.2) is aware of the artifact and will dispatch it under the priority discipline.
4. **In-flight** — at least one chunk of the artifact has been transmitted across the link, and not all transmitted chunks have been acknowledged. The artifact’s freshness vector now reflects in-flight position.
5. **Partially acknowledged** — some chunks have been confirmed received by the counterparty; others are outstanding. The artifact is between in-flight and durable; downstream decisions that read this artifact are flagged as operating against an evolving state (§6.13).
6. **Durable** — all chunks have been confirmed received and reassembled; custody is closed on the receiving side and the artifact is a complete part of the mission state at the converging authority.
7. **Disputed** — the two sides of the link disagree about the artifact’s lifecycle state (for example, the edge believes the artifact is *durable* but the converging authority’s custody log does not record it). The disputed state triggers arbitration per §6.12.
8. **Convergence-pending** — the artifact is under active state-convergence arbitration and is not yet re-

solvable. Downstream personas may not consume an artifact in this state without operator-arbitrated authorization.

Transition invariants. The lifecycle is governed by semantic invariants that the substrate enforces:

- Lifecycle state advances monotonically through the ordered sequence above, except via explicit roll-back (see below). The substrate refuses any transition that would skip a required intermediate state.
- Causal sequence IDs are monotonic within a mission scope. The substrate refuses to assign a causal sequence ID that would create an inversion in the mission's causal ordering.
- Custody handoff between sides preserves lineage integrity. When a custody object transitions from edge custody to shore custody, the lineage chain is extended, not rewritten; both sides' custody logs agree on the lineage at the handoff point.
- A *durable* artifact cannot become *disputed* without a governance-state arbitration entry being created. The substrate refuses silent disagreement-to-dispute transitions; every dispute is governance-visible.

Arbitration states. When the two sides disagree on a custody object's state, the artifact enters the *disputed* lifecycle state and a governance-state arbitration entry is created per §6.12. Resolution of the arbitration emits a new lifecycle state with arbitration metadata embedded in the custody object — the resolution rule applied, the operator-arbitration decision (if any), and the time of resolution. The artifact's lineage records the arbitration as part of its history.

Rollback conditions. Operator revocation (per §6.10) MAY require rolling back a custody object to an earlier lifecycle state. Rollback is recorded as a lineage event — the rollback's authority provenance is itself recorded, including the revocation token and the operator's stated reason — and the artifact's prior state is preserved in the replay-log binding such that the prior lifecycle history is recoverable. Rollback is not a destructive operation; it is a forward-additive event that produces a new lifecycle position.

Deterministic replay coupling. Every custody-object transition is replayable. The replay log of §6.9 plus the custody object's replay binding together suffice to regenerate the artifact's full lifecycle history. The substrate guarantees that the regenerated history is bit-identical to the original history; divergence is a flagged defect.

6.12 Governance-State Arbitration

The orchestration substrate's behavior under conflict and uncertainty is governed by an arbitration discipline that determines when the substrate halts, how it signals halts to the operator, and how operator decisions resume the substrate's autonomous progression. This subsection formalizes that discipline at the semantic level. Implementation specifics — including the internal algorithms that detect conflicts, the data structures that record arbitration entries, the operator-UI mechanisms that surface arbitration to the operator, and the wire format of governance events — are matters of engineering specification outside this paper's scope.

Operator arbitration mode. When the substrate cannot deterministically resolve a state-convergence conflict within the operator-specified rule envelope of §6.3, it does not silently pick a winner. It enters arbitration mode: the conflict is recorded as a governance-state arbitration entry; the affected custody objects are marked *convergence-pending* per §6.11; and the operator is presented with the conflict in a form that supports a deterministic decision. The presentation includes the conflicting writes, the causal ordering established by C-2,

the resolution rules that were attempted, and the reason each attempted rule did not apply.

Uncertainty windows. Periods during which the orchestration substrate has incomplete information about counterparty state are explicitly bracketed as *uncertainty windows*. An uncertainty window has a defined start marker (the substrate noticed that information about the counterparty is no longer current — link drop, expired freshness threshold, missing acknowledgement) and an eventual end marker (the substrate received fresh information closing the window). Inside an uncertainty window, the substrate operates with reduced autonomy: actions that would depend on counterparty state are deferred or escalated per the operator’s policy.

Stale-state confidence. Decisions made by the substrate, or supported by the substrate for an operator decision, carry a confidence indicator derived from the freshness vector of the input artifacts. Below-threshold confidence triggers operator escalation rather than autonomous action: the substrate produces a recommendation but does not act on it; the operator must explicitly admit the recommendation before the substrate proceeds. Confidence thresholds are operator-configurable per persona contract and per mission domain.

Conflict pause conditions. The substrate halts autonomous progression and waits for operator arbitration under a defined set of conditions:

- *Detected write-conflict that exceeds the resolver-rule envelope* — the §6.3 convergence layer cannot determine a winner within the operator-specified rule set.
- *Freshness-vector incompatibility* — two writes have freshness vectors that the §6.13 freshness model cannot order with confidence.
- *Lineage-integrity break* — a custody object’s lineage chain has a discontinuity (a referenced predecessor is unknown, a lineage version conflicts with the recorded chain).
- *Token-scope violation by an upstream persona* — an action arrived bearing custody from an upstream persona that exceeded its contract scope. The substrate refuses the downstream action and surfaces the upstream violation.
- *Suspected adversarial state injection* — an artifact arrives with custody markers that are internally inconsistent (the artifact’s lineage references a predecessor whose hash does not match the lineage record). The substrate refuses the artifact and surfaces the integrity break.

Deterministic halt semantics. When the substrate halts, the halt is itself a deterministic action: the same conflict, the same governance state, and the same operator-policy configuration produce the same halt at the same point. Halts are replayable through the §6.9 log; this property is what makes arbitration auditable. Operator resolutions — the decisions that resume the substrate from a halt — are themselves recorded as governance events that the substrate then deterministically resumes from. The substrate does not improvise resolutions; it executes operator decisions and records both the decision and its provenance.

Token-revoke governance. Token revocation (per §6.10) is a governance event with arbitration consequences. In-flight actions under the revoked token enter conflict-pause for safe-point termination; downstream artifacts whose custody chains depend on outputs produced under the revoked token may enter roll-back per §6.11. The arbitration discipline records the revocation, the in-flight actions that were halted, the downstream artifacts that were rolled back, and the operator’s stated reason for the revocation.

Operator escalation thresholds. Beyond hard conflicts, the substrate proactively surfaces certain state-of-

affairs to the operator without waiting for a halt-triggering condition:

- *Confidence drift over a window* — the running confidence on a class of decisions has drifted below an operator-configured threshold, signalling that the substrate is operating progressively closer to its uncertainty bounds.
- *Lineage-chain depth exceeding threshold* — an artifact’s lineage chain has grown deeper than the operator’s policy permits without explicit operator-acknowledged consolidation.
- *Rate-of-disputed-artifacts spike* — the substrate is producing disputed-state artifacts at a rate higher than the operational baseline, signalling a possible link-state or counterparty-state anomaly.
- *Freshness-vector divergence between sides* — the two sides’ freshness vectors have diverged beyond the operator’s configured tolerance, signalling that the mission state is becoming progressively non-converged.

These escalations are not halts; they are operator-visible signals that the substrate is approaching the boundary of its operator-bounded autonomy envelope. The operator may respond by adjusting policy, by halting the substrate explicitly, or by accepting the current operating regime.

6.13 Freshness-Aware Mission Semantics

The freshness telemetry of §6.4 is grounded in a behavioral discipline that governs how the substrate exposes mission state’s age to the operator and to downstream personas. This subsection formalizes that discipline at the semantic level. Implementation specifics — including the arithmetic of freshness-vector composition, the timestamping mechanism, the operator-UI rendering of trust indicators, and the threshold-evaluation algorithms — are matters of engine